Architecture Recommendation: Consolidated Report System

Document: Architecture Design
Date: 2026-04-09
For: Ward (Backend) + Frontend Team
Status: Proposal for Review & Sign-Off

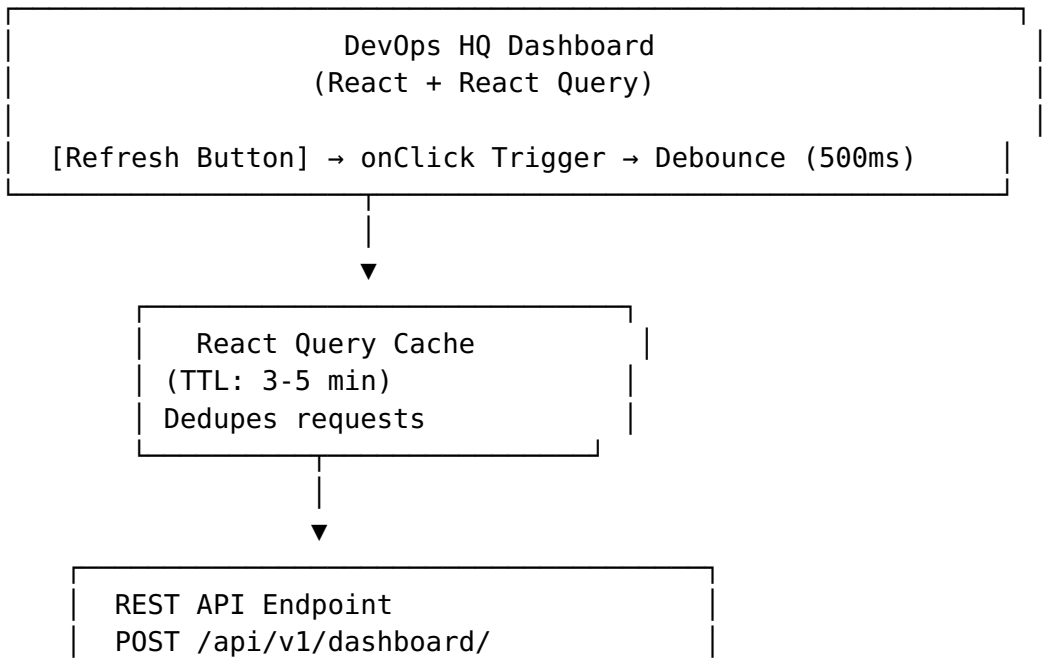
Executive Summary

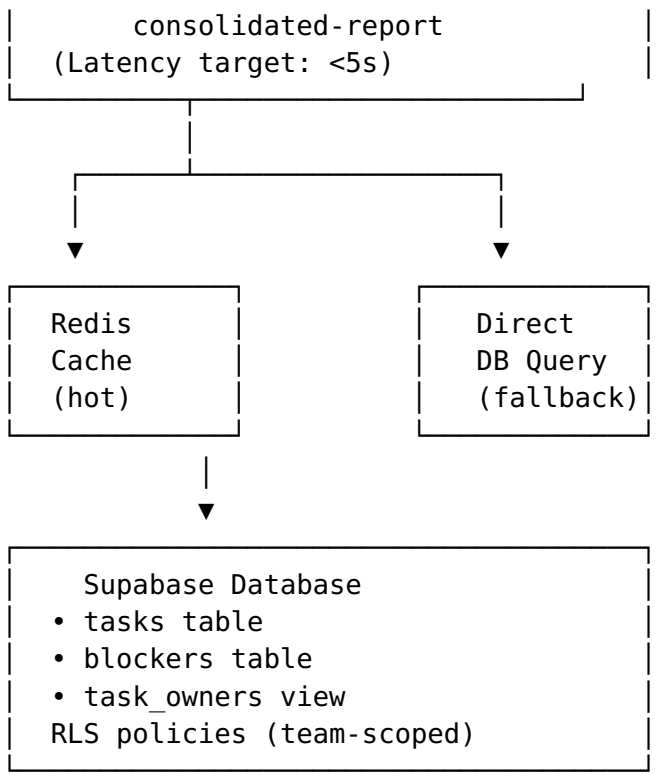
Recommendation: Hybrid API + CLI architecture with React Query client-side caching.

Key decision points: - API-first (REST) for real-time UI updates - CLI fallback for scheduled/batch reports - Redis caching for <5s latency - Supabase RLS for data isolation - WebSocket (Phase 2) for live updates

---

Architecture Diagram





Data Flow: Detailed

```
TRIGGER: User clicks [Refresh ↻]
    ↓
DEBOUNCE: 500ms (prevent double-clicks)
    ↓
CACHE CHECK: React Query cache hit?
    ├─ YES → Return cached data, update UI, skip API call
    └─ NO → Continue
    ↓
API CALL: POST /api/v1/dashboard/consolidated-report
    Headers: Authorization (JWT), Content-Type: application/json
    Body: { filters: { team?: string, status?: string, owner?:
string } }
    Timeout: 10s
    ↓
BACKEND PROCESSING:
    1. Verify user authorization (RLS via Supabase)
    2. Query tasks, blockers, owners (potentially cached in Redis)
    3. Aggregate: count by status, risk, owner
    4. Detect blockers: tasks with status = 'blocked'
    5. Calculate risk score (auto or manual field)
    6. Sort blockers by risk + age
    7. Return JSON response
    ↓
RESPONSE: { summary, blockers, tasks, generatedAt }
    ↓
CACHE STORE: React Query caches with 3-5 min TTL
    ↓
```

```
RENDER: Dashboard updates with new data
    ├─ Summary cards re-render
    ├─ Blockers section updates
    └─ Task table refresh
    ↓
USER ACTIONS:
    ├─ Filter by status/owner
    ├─ Sort table
    ├─ Export (CSV/PDF/Slack)
    └─ Inline actions (Unblock, Escalate)
```

Technology Stack

Frontend

```
React 18+ (with TypeScript)
├─ React Query v5 (data fetching + caching)
├─ TanStack Table (sorting + filtering)
├─ Tailwind CSS (styling)
├─ csv-download (CSV export)
├─ jsPDF (PDF generation)
└─ Axios (HTTP client)
```

Backend (Ward's Domain)

```
Node.js / Deno (assuming)
├─ Express or Hono (HTTP framework)
├─ Supabase (PostgreSQL + RLS)
├─ Redis (optional, for caching)
├─ TypeScript
└─ Zod or joi (schema validation)
```

Infrastructure

```
Supabase (hosted PostgreSQL)
├─ Connection pooling (PgBouncer)
├─ RLS policies (auth-based)
└─ Real-time subscriptions (WebSocket, Phase 2)

Redis (optional, Phase 1.5+)
├─ Cached responses (JSON)
├─ TTL: 3-5 min
└─ Invalidation: on task/blocker updates

CDN (Phase 2)
├─ Cache static report templates
└─ Edge functions for report generation
```

API Contract

Endpoint Definition

```
POST /api/v1/dashboard/consolidated-report
Authorization: Bearer <JWT_TOKEN>
Content-Type: application/json

Request Body:
{
  "filters": {
    "team": "platform" | null,
    "status": "blocked" | "in-progress" | "proposed" | "completed"
| null,
    "owner": "ward@example.com" | null,
    "riskLevel": "high" | "medium" | "low" | null,
    "limit": 50,
    "offset": 0
  }
}

Response (200 OK):
{
  "summary": {
    "total": 25,
    "completed": 12,
    "inProgress": 5,
    "proposed": 3,
    "blocked": 2,
    "atRiskCount": 3,
    "velocity": 2.4  // completed per week
  },

  "blockers": [
    {
      "id": "blocker-104",
      "taskId": "task-104",
      "taskTitle": "API Schema Design",
      "reason": "Backend team hasn't delivered schema",
      "owner": "ward@example.com",
      "ownerName": "Ward",
      "riskLevel": "high",
      "ageHours": 72,
      "suggestedAction": "escalate"
    },
    {
      "id": "blocker-087",
      "taskId": "task-087",
      "taskTitle": "Design Approval",
      "reason": "Pending stakeholder sign-off",
      "owner": "jamie@example.com",
```

```
      "ownerName": "Jamie",
      "riskLevel": "medium",
      "ageHours": 24,
      "suggestedAction": "ping"
    }
  ],

  "tasks": [
    {
      "id": "task-104",
      "title": "API Schema Design",
      "status": "blocked",
      "owner": "ward@example.com",
      "ownerName": "Ward",
      "riskLevel": "high",
      "percentComplete": 0,
      "updatedAt": "2026-04-06T14:32:00Z",
      "blockerCount": 1
    },
    {
      "id": "task-103",
      "title": "DB Migration",
      "status": "in-progress",
      "owner": "morgan@example.com",
      "ownerName": "Morgan",
      "riskLevel": "medium",
      "percentComplete": 60,
      "updatedAt": "2026-04-09T18:45:00Z",
      "blockerCount": 0
    },
    {
      "id": "task-102",
      "title": "Auth Module",
      "status": "completed",
      "owner": "selym@example.com",
      "ownerName": "Selym",
      "riskLevel": "low",
      "percentComplete": 100,
      "updatedAt": "2026-04-05T10:15:00Z",
      "blockerCount": 0
    }
  ],

  "generatedAt": "2026-04-09T19:00:00Z",
  "cacheHit": false,
  "executionTimeMs": 1240
}

Error (400 Bad Request):
{
  "error": "INVALID_FILTER",
  "message": "Invalid status filter",
```

```
  "code": "E400_BAD_REQUEST"
}

Error (401 Unauthorized):
{
  "error": "UNAUTHORIZED",
  "message": "JWT expired or invalid",
  "code": "E401_UNAUTHORIZED"
}

Error (500 Internal Server Error):
{
  "error": "QUERY_TIMEOUT",
  "message": "Database query exceeded 10s timeout",
  "code": "E500_TIMEOUT"
}
```

Database Schema (Supabase)

Required Tables/Views

```sql
-- Existing task table (assumed)
CREATE TABLE tasks (
  id UUID PRIMARY KEY,
  title TEXT NOT NULL,
  status TEXT NOT NULL, -- 'proposed' | 'in-progress' |
        'completed' | 'blocked'
  assigned_to UUID, -- FK to auth.users
  risk_level TEXT, -- 'low' | 'medium' | 'high' (auto-calculated
        or manual)
  percent_complete INT DEFAULT 0,
  updated_at TIMESTAMP DEFAULT now(),
  created_at TIMESTAMP DEFAULT now(),
  team_id UUID -- for multi-team isolation
);

-- Blockers table (NEW)
CREATE TABLE blockers (
  id UUID PRIMARY KEY,
  task_id UUID NOT NULL REFERENCES tasks(id),
  reason TEXT NOT NULL,
  owner_id UUID REFERENCES auth.users,
  created_at TIMESTAMP DEFAULT now(),
  resolved_at TIMESTAMP NULL,
  risk_level TEXT -- 'low' | 'medium' | 'high'
);

-- Task owners view (optional, for easier queries)
CREATE VIEW task_owners_view AS
```

```sql
SELECT
  t.id,
  t.title,
  t.status,
  u.email as owner_email,
  u.user_metadata ->> 'full_name' as owner_name,
  t.risk_level,
  COUNT(b.id) as blocker_count,
  MAX(b.created_at) as oldest_blocker_age
FROM tasks t
LEFT JOIN auth.users u ON t.assigned_to = u.id
LEFT JOIN blockers b ON t.id = b.task_id AND b.resolved_at IS NULL
GROUP BY t.id, u.email, u.user_metadata;

-- RLS Policy Example
ALTER TABLE tasks ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users see tasks in their team"
ON tasks
FOR SELECT
USING (
  auth.uid() = assigned_to OR
  team_id IN (
    SELECT team_id FROM team_members WHERE user_id = auth.uid()
  )
);
```

Caching Strategy

Client-Side (React Query)

```javascript
const {
  data,
  isLoading,
  refetch,
  isFetching
} = useQuery({
  queryKey: ['consolidated-report', filters],
  queryFn: fetchConsolidatedReport,
  staleTime: 3 * 60 * 1000, // 3 minutes
  cacheTime: 5 * 60 * 1000, // Keep in memory 5 min
  onError: (error) => showErrorToast(error.message),
});
```

Server-Side (Redis)

```javascript
// In API endpoint
const cacheKey = `report:${userId}:${JSON.stringify(filters)}`;
```

```
const cached = await redis.get(cacheKey);

if (cached) {
  return { ...JSON.parse(cached), cacheHit: true };
}

// Generate fresh report
const report = aggregateReport(filters);
await redis.setex(cacheKey, 300,
        JSON.stringify(report)); // 5 min TTL

return { ...report, cacheHit: false };
```

Invalidation Strategy

- Cache invalidates when:
  - Task status changes
  - Blocker is created/resolved
  - Task owner changes
  - Risk level changes
  - Manual cache clear (admin action)
- Use message queue or webhook to trigger invalidation

Error Handling & Resilience

Timeouts

- API endpoint timeout: 10s (server-side)
- React Query retry: 3 attempts with exponential backoff
- Fallback: Show cached data if >2 failures

Graceful Degradation

```
Scenario: API down or slow
├─ First click: Wait 10s, show error toast
├─ Second click: Retry with cached data (if <30 min old)
└─ Third click: Show "offline mode" with last known state
```

Rate Limiting

- 30 requests per user per minute (typical SaaS)
- 429 response with `Retry-After` header
- Implement client-side debounce (500ms) to prevent

Performance Targets

Metric	Target	Note
API Response Time	<5s (cold), <1s (cached)	50-100 tasks
Dashboard Render	<500ms	After API returns
Time to Interactive	<3s	Full page load
Cache Hit Rate	80%+	Most refreshes hit cache
P95 Latency	<8s	95th percentile request

Phase Breakdown

Phase 1: MVP (Week 1)

- [x] Design complete (this doc)
- [ ] Backend: Basic API endpoint + direct DB query
- [ ] Frontend: Component scaffold + mock data
- [ ] Cache: React Query only, no Redis yet
- [ ] Export: CSV only

Phase 1.5: Production Ready (Week 1 end)

- [ ] Redis caching added
- [ ] Load testing (100 concurrent users)
- [ ] Error handling & retry logic
- [ ] PDF export

Phase 2: Enhanced (Week 2+)

- [ ]

- [ ] WebSocket real-time updates
- [ ] Slack integration
- [ ] Scheduled reports (cron)
- [ ] Analytics dashboard (trends over time)
- [ ] Auto-refresh toggle

Security Considerations

1. **Authentication:** JWT token validation on every API request
2. **Authorization:** RLS policies at database level (Supabase)
3. **Data Isolation:** Queries filtered by user's team
4. **Input Validation:** Zod schema validation on all filters
5. **Rate Limiting:** 30 req/user/min
6. **CORS:** Restrict to trusted origins only
7. **SQL Injection:** Use parameterized queries (ORM/prepared statements)

Monitoring & Observability

Metrics to Track

- API response time (avg, p95, p99)
- Cache hit rate
- Error rate (4xx, 5xx)
- Query execution time
- User session duration
- Refresh button click frequency

Logging

- All API requests (method, path, status, latency)
- Cache misses + queries that triggered them
- Errors with stack trace
- Data mutations (task/blocker changes)

Alerting

- API latency >10s for 5+ min → alert
- Error rate >5% → alert
- Redis connection lost → alert

- Database query timeout → alert

Open Questions for Ward

1. **Risk Level Calculation:** Auto-computed from task age/priority, or manual field?
2. **Blocker Detection:** Is “blocked” status set manually, or auto-detect from dependencies?
3. **Owner Assignment:** Single owner or multiple? How to surface in report?
4. **Real-Time Updates:** Can we use Supabase Realtime subscriptions, or stick to polling?
5. **Data Retention:** How long to keep historical reports? Archive old ones?
6. **Multi-Team Support:** Should report scope to a single team, or org-wide?

Recommendation Summary

 Use hybrid API + CLI - REST API for real-time UI (primary) - CLI for scheduled/batch jobs (secondary)

 Cache aggressively - React Query (client) + Redis (server) - 3-5 min TTL, invalidate on writes

 Target <5s response time - Direct DB query (optimized indexes) - Redis cache for hot paths - CDN for static assets

 Start simple, scale later - MVP: API + React Query - Phase 1.5: Add Redis - Phase 2: WebSocket + advanced features

---

Approval Required From: - [ ] Ward (Backend Architecture) - [ ] Frontend Lead (UI Component Approach) - [ ] DevOps (Infrastructure/Scaling Concerns)